

PROJECT MANUAL

RESEARCH PAPER RETRIEVAL-AUGMENTED GENERATION (RAG) SYSTEM

Student Name	Jagan
Registration Number	231FA23087
Email	student@example.com
GitHub Repository	https://github.com/jagan/research-paper-rag-system

1. Project Overview

The Research Paper RAG System is an intelligent document-based question-answering application engineered to analyze academic literature and synthesize context-grounded answers with verifiable page-level source citations. Instead of generic language model generation that suffers from confabulation and hallucination, the system reconstructs complex multi-column scientific layouts, performs section-bounded semantic chunking, executes dense vector retrieval, and enforces strict factual provenance for all generated claims.

The project pairs a high-performance Python backend with an interactive Streamlit web interface and a CLI. This modular separation provides a structured pipeline for layout-aware document ingestion, ChromaDB vector indexing, multi-provider LLM synthesis (Gemini, OpenAI, Groq), and local extractive offline processing.

2. Project Objective

The primary objective is to allow users to upload research papers and extract factual, verifiable answers to essential research questions including the paper's overarching objective, methodology, benchmark datasets, major empirical findings, and stated limitations. Every synthesized answer must be explicitly grounded in the source text with precise **[Page X, Section Y]** citations.

3. How the System Works

- 1. PDF Ingestion:** The user uploads an academic research paper in PDF format.
- 2. Layout-Aware Parsing:** PyMuPDF extracts text blocks and coordinates, sorting columns to prevent reading crossover.
- 3. Section Detection:** Academic section headers (Abstract, Introduction, Methodology, etc.) are detected and cataloged.
- 4. Section-Bounded Chunking:** Text is split into 800-1,000 character windows preserving section context and page numbers.
- 5. Dense Vector Embedding:** BAAI/bge-small-en-v1.5 embeddings are generated via FastEmbed ONNX and stored in ChromaDB.
- 6. Semantic Retrieval & Boosting:** User queries retrieve top-k chunks with priority boosts for target academic sections.
- 7. Grounded Generation & Citation:** LLM synthesizes context-confined answers with verifiable page-level citations.

4. Technologies Used

The project uses **Python 3.10** for backend processing and RAG orchestration, **Streamlit** for the interactive web interface, **PyMuPDF (fitz)** for layout-aware PDF parsing and column ordering, **ChromaDB** for persistent and in-memory vector storage, **FastEmbed (BGE-small ONNX)** for lightweight CPU semantic embeddings, **Google Gemini / OpenAI / Groq SDKs** for conversational grounded generation, and **Pytest** for automated test verification.

5. Retrieval Process - Simple Explanation

When a user asks a question (such as *'What datasets were used?'*), the system converts the query into a high-dimensional vector and performs cosine similarity search across the paper's indexed chunks. Chunks originating from relevant sections (e.g., *'Experiments & Datasets'*) receive an algorithmic relevance boost. The top-ranked passages are combined into a provenance-labeled prompt that strictly instructs the language model to answer solely using the provided excerpts and cite the corresponding page numbers.

6. Application Workflow

- Launch the RAG application (Streamlit web dashboard or CLI).
- Upload a research paper PDF or click 'Load Sample Paper (DeepScale)'.
- Review extracted document metrics (total pages, detected sections, total chunks).
- Select an LLM provider (Google Gemini, OpenAI, Groq, or Local Offline Mode).
- Click a 1-click research preset (Objective, Methodology, Datasets, Findings, Limitations).
- View grounded answer and expand 'Verified Sources & Evidence' to examine citations and excerpts.

7. Project Architecture

The system is built on a modular pipeline architecture with clean separation between ingestion, indexing, retrieval, and generation:

- **Loader Layer (rag/loader.py):** Block-coordinate PDF parser handling multi-column formats and section boundaries.
- **Chunker Layer (rag/chunker.py):** Semantic sliding-window chunker tagging every chunk with page and section metadata.
- **Vector Store Layer (rag/vector_store.py):** ChromaDB client with FastEmbed embeddings and BM25 fallback.
- **Generator Layer (rag/generator.py):** Grounded generation engine with strict anti-hallucination prompts.
- **Citation Layer (rag/citation.py):** Regex parser verifying [Page X, Section Y] tags against physical text chunks.
- **Presentation Layer (app.py & cli.py):** Streamlit web dashboard and terminal interface.

8. Key Advantages

- **Layout-Aware Ingestion:** Accurately separates columns in standard academic papers (ArXiv, IEEE, NeurIPS).
- **Zero Hallucination:** Strict context-grounded prompting ensures only verifiable facts are synthesized.
- **Verifiable Source Citations:** Every claim is attributed to a specific page number and section heading.
- **Offline Capability:** Operates completely offline with local ONNX embeddings and rule-based extractive synthesis.
- **1-Click Academic Inquiries:** Instant answers for Objective, Methodology, Datasets, Findings, and Limitations.
- **Transparent Inspection:** Users can view detected document outlines and inspect underlying vector chunks.

9. GitHub Source Code

The complete project source code, unit test suite, and documentation are available at:

GitHub Repository: <https://github.com/jagan/research-paper-rag-system>

Student Name: Jagan | **Registration No.:** 231FA23087

Email: student@example.com